

[Assignment VIZ]

Problem 1: Transpose of downsampling. Consider the downsampling operator $\mathcal{T}: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{(m/2) \times (n/2)}$, defined as the average pool with a 2×2 kernel and stride 2. For the sake of simplicity, assume m and n are even. Describe the action of $\mathcal{T}^\top$. More specifically, describe how to compute $\mathcal{T}^\top(Y)$ for any $Y \in \mathbb{R}^{(m/2) \times (n/2)}$.

Clarification. The downsampling operator $\mathcal{T}$ is a linear operator (why?). Therefore, $\mathcal{T}$ has a matrix representation $A \in \mathbb{R}^{(mn/4) \times (mn)}$ such that

$$\mathcal{T}(X) = (A(X.\text{reshape}(mn))).\text{reshape}(m/2, n/2)$$

for all $X \in \mathbb{R}^{m \times n}$. The adjoint $\mathcal{T}^\top$ has two equivalent definitions. One definition is

$$\mathcal{T}^\top(Y) = (A^\top(Y.\text{reshape}(mn/4))).\text{reshape}(m, n)$$

for all $Y \in \mathbb{R}^{(m/2) \times (n/2)}$. Another is

$$\sum_{i=1}^{m/2} \sum_{j=1}^{n/2} Y_{ij} (\mathcal{T}(X))_{ij} = \sum_{i=1}^m \sum_{j=1}^n (\mathcal{T}^\top(Y))_{ij} (X)_{ij}$$

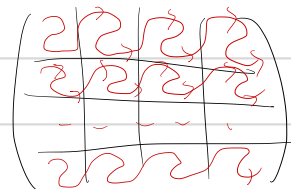
for all $X \in \mathbb{R}^{m \times n}$ and $Y \in \mathbb{R}^{(m/2) \times (n/2)}$.

Hint. To spoil the suspense, $\mathcal{T}^\top$ is a constant times the nearest neighbor upsampling. Explain why in your answer.

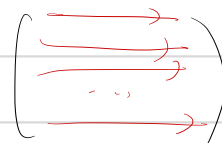
(손쉬운 시각화)

$\mathbb{R}^{m \times n}$: $\{e_{11}, e_{12}, e_{21}, e_{22}, e_{13}, e_{14}, e_{23}, e_{24}, \dots\}$

$\dots\} = \mathcal{B}$



$\mathbb{R}^{m/2 \times n/2}$: $\{e_{11}, e_{12}, \dots\} = \mathcal{L}$



◦ $\mathcal{T}$: (nearest map? $\Rightarrow \mathcal{T}(C \cup N) = \mathcal{T}(C \cup N)$, $\mathcal{T}(C \cup M) = C \cup \mathcal{T}(M)$. obvious

$\Rightarrow$ (손쉬운 시각화)

$$[\mathcal{T}]_{\mathcal{L}}^{\mathcal{B}} = \begin{pmatrix} \begin{matrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{matrix} & \begin{matrix} 0 & 0 & 0 & 0 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{matrix} & \dots & \begin{matrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{matrix} \end{pmatrix} \in \mathbb{R}^{\frac{mn}{4} \times mn}$$

, $\mathcal{B}, \mathcal{L}$: orthonormal basis on

$$\Rightarrow [\mathcal{T}^\top]_{\mathcal{B}}^{\mathcal{L}} = \begin{pmatrix} \begin{matrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{matrix} & \begin{matrix} 0 & 0 & 0 & 0 \end{matrix} & \dots & \begin{matrix} 0 \\ 0 \\ 0 \\ 0 \end{matrix} \\ \begin{matrix} 0 & 0 & 0 & 0 \end{matrix} & \begin{matrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{matrix} & \dots & \begin{matrix} 0 \\ 0 \\ 0 \\ 0 \end{matrix} \\ \dots & \dots & \dots & \dots \\ \begin{matrix} 0 & 0 & 0 & 0 \end{matrix} & \begin{matrix} 0 & 0 & 0 & 0 \end{matrix} & \dots & \begin{matrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{matrix} \end{pmatrix} \in \mathbb{R}^{mn \times \frac{mn}{4}}$$

$$\Rightarrow [\mathcal{T}^\top]_{\mathcal{B}}^{\mathcal{L}} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1, \frac{n}{2}} \\ \vdots & \vdots & \ddots & \vdots \\ x_{\frac{m}{2}, 1} & x_{\frac{m}{2}, 2} & \dots & x_{\frac{m}{2}, \frac{n}{2}} \end{pmatrix} = \begin{pmatrix} x_{11} & x_{11} & \dots & x_{11} \\ x_{12} & x_{12} & \dots & x_{12} \\ \vdots & \vdots & \ddots & \vdots \\ x_{\frac{m}{2}, 1} & x_{\frac{m}{2}, 1} & \dots & x_{\frac{m}{2}, 1} \\ x_{\frac{m}{2}, 2} & x_{\frac{m}{2}, 2} & \dots & x_{\frac{m}{2}, 2} \\ \vdots & \vdots & \ddots & \vdots \\ x_{\frac{m}{2}, \frac{n}{2}} & x_{\frac{m}{2}, \frac{n}{2}} & \dots & x_{\frac{m}{2}, \frac{n}{2}} \end{pmatrix}$$

($\frac{1}{4} \times$ nearest neighbour upsampling)

Problem 2: Nearest neighbor upsampling. How is the nearest neighbor upsampling operator an instance of transpose convolution? Specifically, describe how

```
layer = nn.Upsample(scale_factor=r, mode='nearest')
```

can be equivalently represented by

```
layer = nn.ConvTranspose2d(...)
layer.weight.data = ...
```

with ... appropriately filled in.

Problem 1의 m 은 r 배로 늘었다,

$$\begin{pmatrix} x_{11} & \dots & x_{1n} \\ \vdots & & \vdots \\ x_{m1} & \dots & x_{mn} \end{pmatrix} \Rightarrow \begin{pmatrix} x_{11} \dots x_{1n} & & \\ x_{r1} \dots x_{rn} & & \\ & \ddots & \\ & & x_{m1} \dots x_{mn} \end{pmatrix}$$

은 r 배로 늘었다, r 배로 늘었다.

이제 r 배로 Problem 1의 r 배로 늘었다, r 배로 늘었다,

$$\begin{pmatrix} x_{11} \\ \vdots \\ x_{1n} \\ x_{21} \\ \vdots \\ x_{2n} \\ \vdots \\ x_{m1} \\ \vdots \\ x_{mn} \end{pmatrix} \Leftarrow A \begin{pmatrix} x_{11} \\ \vdots \\ x_{mn} \end{pmatrix}, \quad A \in \mathbb{R}^{r \cdot mn \times mn} \text{ 이다.}$$

$$\text{layer.weight.data} (=A) \leftarrow \begin{pmatrix} \vdots & & 0 & & 0 \\ \vdots & & & & \\ \vdots & & & & \\ 0 & & & & 0 \\ & \ddots & & & \\ 0 & & & & \vdots \end{pmatrix}$$

$$\text{layer} \leftarrow \text{nn.ConvTranspose2d}((1,1), (r,r), 0, \text{bias}=\text{False})$$

in out kernel, stride padding

이제 channel이 r 배로 늘었다 $\text{nn.ConvTranspose2d}((C,C), (r,r), 0, \text{bias}=\text{False})$ 이,

layer.weight.data 이

$$\Leftarrow \begin{pmatrix} \begin{pmatrix} 0 & A & 0 \end{pmatrix} \\ \vdots \\ \begin{pmatrix} 0 & 0 & A \end{pmatrix} \end{pmatrix} \begin{pmatrix} 0 & & \\ & \ddots & \\ & & 0 \end{pmatrix} \begin{pmatrix} 0 & & \\ & \ddots & \\ & & 0 \end{pmatrix} \begin{pmatrix} 0 & & \\ & \ddots & \\ & & 0 \end{pmatrix} \end{pmatrix} C, \quad A \leftarrow$$

은 r 배로 늘었다,

Problem 3: The true softmax. Define

$$\nu_\beta(x) = \frac{1}{\beta} \log \sum_{i=1}^n \exp(\beta x_i).$$

Clearly, $\nu_\beta: \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable. Show that

(a) $\nu_\beta(x) \rightarrow \max\{x_1, \dots, x_n\}$ as $\beta \rightarrow \infty$.

최대값을 갖는 x_i $x_i = \max\{x_1, \dots, x_n\}$ 찾기.

$$\nu_\beta(x) = \frac{1}{\beta} \log (e^{\beta x_1} (1 + e^{\beta(x_2-x_1)} + \dots + e^{\beta(x_n-x_1)}))$$

$$= x_1 + \frac{1}{\beta} \log (1 + e^{\beta(x_2-x_1)} + \dots + e^{\beta(x_n-x_1)})$$

$$\beta \rightarrow \infty; 1 \leq 1 + e^{\beta(x_2-x_1)} + \dots + e^{\beta(x_n-x_1)} \leq n$$

$$\nu_\beta(x) \rightarrow x_1$$

(b) $\nabla \nu_1 = \mu$, where μ is the softmax function.

$$\nu(x) = \log \sum_{i=1}^n \exp(x_i)$$

$$\Rightarrow \frac{\partial \nu}{\partial x_i} = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} \Rightarrow \nabla \nu = \frac{1}{\sum_{j=1}^n \exp(x_j)} (\exp(x_1), \dots, \exp(x_n)) = \text{softmax}(x) //$$

(c) If $i_{\max} = \operatorname{argmax}_{1 \leq i \leq n} x_i$ is uniquely defined, then $\nabla \nu_\beta(x) \rightarrow e_{i_{\max}}$ as $\beta \rightarrow \infty$, where $\{e_1, \dots, e_n\}$ is the standard basis of $\mathbb{R}^n$.

$$\frac{\partial \nu}{\partial x_i} = \frac{1}{n} \frac{\exp(\beta x_i)}{\sum_{j=1}^n \exp(\beta x_j)} = \frac{\exp(\beta x_i)}{\sum_{j=1}^n \exp(\beta x_j)} = \frac{e^{\beta(x_i - x_{\max})}}{\sum_{j=1}^n e^{\beta(x_j - x_{\max})}}$$

$$\sum_{j=1}^n e^{\beta(x_j - x_{\max})} = 1 + \sum_{i \neq i_{\max}} e^{\beta(x_i - x_{\max})} < 0 \text{ ; } i_{\max} = \operatorname{argmax}_{1 \leq i \leq n} x_i \rightarrow 1 \text{ if } \beta \rightarrow \infty$$

$$\text{if } i = i_{\max} \quad e^{\beta(x_i - x_{\max})} = 1$$

$$\text{else} \quad e^{\beta(x_i - x_{\max})} \rightarrow 0 \text{ if } \beta \rightarrow \infty$$

$$\Rightarrow \nabla \nu_\beta(x) \rightarrow e_{i_{\max}} //$$

Problem 4: Generalized inverse transform sampling. Let $F: \mathbb{R} \rightarrow [0, 1]$ be the CDF of a random variable, and let $U \sim \text{Uniform}([0, 1])$. If F is strictly increasing and therefore invertible, then $F^{-1}(U)$ is a random variable with CDF F , because

$$\mathbb{P}(F^{-1}(U) \leq t) = \mathbb{P}(U \leq F(t)) = F(t).$$

When F is not necessarily invertible, the *generalized inverse* of F is $G: (0, 1) \rightarrow \mathbb{R}$ with

$$G(u) = \inf\{x \in \mathbb{R} \mid u \leq F(x)\}.$$

Show $G(U)$ with is a random variable with CDF F .

Hint. Use the fact that F is right-continuous, i.e., $\lim_{h \rightarrow 0^+} F(x+h) = F(x)$ for all $x \in \mathbb{R}$, and that $\lim_{x \rightarrow -\infty} F(x) = 0$.

$t \in \mathbb{R}$ and $\{u: G(u) \leq t\} = \{u: u \leq F(t)\}$ are both sets of u .

$$\textcircled{1} \quad G(u) \leq t \Rightarrow u \leq F(t)$$

$$\inf\{x \in \mathbb{R} : u \leq F(x)\} \leq t \Rightarrow A = \inf\{x \in \mathbb{R} : u \leq F(x)\} \leq t \quad A \in \mathbb{R}, t \geq 0 \quad u \leq F(A) \leq F(t) \quad (F \text{ is monotone})$$

$$\Rightarrow \varepsilon > 0; \quad u \leq F(A) \leq F(t), \quad (\text{right-continuity of CDF})$$

$$\textcircled{2} \quad u \leq F(t) \Rightarrow G(u) \leq t$$

$$t \in \mathbb{R} \quad x \in \mathbb{R} : u \leq F(x) \Rightarrow G(u) = \inf\{x \in \mathbb{R} : u \leq F(x)\} \leq t \quad \square$$

To prove this property
requires some knowledge of
probability measure theory.

Problem 5: f -divergence. Let X and Y be two continuous random variables with densities p_X and p_Y . The f -divergence of X from Y is defined as

$$D_f(X\|Y) = \int f\left(\frac{p_X(x)}{p_Y(x)}\right) p_Y(x) dx,$$

where f is a convex function such that $f(1) = 0$.

(a) Show that $D_f(X\|Y) \geq 0$.

Jensen's inequality $D_f(X\|Y) = \int f\left(\frac{p_X(x)}{p_Y(x)}\right) p_Y(x) dx \geq f\left(\int \frac{p_X(x)}{p_Y(x)} p_Y(x) dx\right) = f\left(\int p_X(x) dx\right) = f(1) = 0$ (f: convex)

(b) Show that $f = -\log t$ and $f = t \log t$ correspond to KL divergence.

$$f = -\log t \Rightarrow D_f(X\|Y) = \int -\log\left(\frac{p_X(x)}{p_Y(x)}\right) p_Y(x) dx = \int p_Y(x) \log\left(\frac{p_Y(x)}{p_X(x)}\right) dx = D_{KL}(Y\|X),$$

$$f = t \log t \Rightarrow D_f(X\|Y) = \int \frac{p_X(x)}{p_Y(x)} \cdot \log\left(\frac{p_X(x)}{p_Y(x)}\right) p_Y(x) dx = \int p_X(x) \log\left(\frac{p_X(x)}{p_Y(x)}\right) dx = D_{KL}(X\|Y),$$

Problem 6: Backprop with convolutions. Consider 1D convolutions with single input and output channels, stride 1, and padding 0. Let $w_1, \dots, w_L$ be convolutional filters with sizes $f_1, \dots, f_L$. Let $A_{w_\ell} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, where $n_\ell = n_{\ell-1} - f_\ell + 1$, be the matrix representing convolution with w_ℓ , i.e., multiplication by A_{w_ℓ} is equivalent to convolution with w_ℓ , for $\ell = 1, \dots, L$. Let $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ be a differentiable activation function. Consider the convolutional neural network

$$\begin{aligned} y_L &= A_{w_L} y_{L-1} + b_L \mathbf{1}_{n_L} \\ y_{L-1} &= \sigma(A_{w_{L-1}} y_{L-2} + b_{L-1} \mathbf{1}_{n_{L-1}}) \\ &\vdots \\ y_2 &= \sigma(A_{w_2} y_1 + b_2 \mathbf{1}_{n_2}) \\ y_1 &= \sigma(A_{w_1} x + b_1 \mathbf{1}_{n_1}), \end{aligned}$$

where $x \in \mathbb{R}^{n_0}$, $b_\ell \in \mathbb{R}$, $\mathbf{1}_{n_\ell} \in \mathbb{R}^{n_\ell}$ is the vector with all entries being 1, and $n_L = 1$. For notational convenience, define $y_0 = x$.

(a) Define

$$v_L = 1, \quad v_\ell = \frac{\partial y_L}{\partial y_\ell} \text{diag}(\sigma'(A_{w_\ell} y_{\ell-1} + b_\ell \mathbf{1}_{n_\ell})) \quad \text{for } \ell = 1, \dots, L-1.$$

Let $C_{v_\ell^\top}$ be the 1D convolutional operator defined by interpreting $v_\ell^\top \in \mathbb{R}^{n_\ell}$ as a convolutional filter for $\ell = 1, \dots, L$. Show that

$$\begin{aligned} \frac{\partial y_L}{\partial y_{L-1}} &= A_{w_L}, \quad \frac{\partial y_\ell}{\partial y_{\ell-1}} = \text{diag}(\sigma'(A_{w_\ell} y_{\ell-1} + b_\ell \mathbf{1}_{n_\ell})) A_{w_\ell} \quad \text{for } \ell = 2, \dots, L-1 \\ \frac{\partial y_L}{\partial w_\ell} &= (C_{v_\ell^\top} y_{\ell-1})^\top \quad \text{for } \ell = 1, \dots, L \\ \frac{\partial y_L}{\partial b_\ell} &= v_\ell \mathbf{1}_{n_\ell} \quad \text{for } \ell = 1, \dots, L. \end{aligned}$$

1D convolutional filter의 공배는 2배

(b) As discussed in homework 1, forming the full matrix A_{w_ℓ} is wasteful and should be avoided. Describe how matrix-vector or vector-matrix products with respect to A_{w_ℓ} or $A_{w_\ell}^\top$ should be used in the forward pass and backpropagation.

이전 forward mode의 경우 y_ℓ 의 값이 0인 경우 무시됨.

이전 backward mode의 경우 $\frac{\partial y_L}{\partial y_\ell}$ 의 값을 A_{w_ℓ} 로 곱함

1D convolutional filter가 곱해지는 구조로, $\frac{\partial y_L}{\partial b_\ell}$ 은 $\frac{\partial y_L}{\partial y_\ell} \text{diag}(\sigma'(y_\ell))$

$$\textcircled{1} y_L = \sum_j (A_{w_L})_{1,j} y_{L-1,j} + b_L \Rightarrow \frac{\partial y_L}{\partial y_{L-1,j}} = (A_{w_L})_{1,j}, \quad \frac{\partial y_L}{\partial y_{L-1}} = A_{w_L}$$

임의의 j 에 대한 $\frac{\partial y_L}{\partial y_{L-1,j}}$ 의 forward mode의 계산은

이것을, $\frac{\partial y_L}{\partial y_{L-1,j}}$ 은 $C_{v_L^\top}$ 라는 1D

$$\textcircled{2} y_{L-1} = \sigma(\sum_j (A_{w_{L-1}})_{1,j} y_{L-2,j} + b_{L-1}) \Rightarrow \frac{\partial y_{L-1}}{\partial y_{L-2,j}} = \sigma'(A_{w_{L-1}})_{1,j} \Rightarrow \frac{\partial y_L}{\partial y_{L-2,j}} = \text{diag}(\sigma') A_{w_{L-1}}$$

convolutional filter의 forward

pass를 통해 y_{L-1} 의 값을 얻는다.

따라서 많은 연산량을 줄일 수 있다.

$$\textcircled{3} \text{if } \frac{\partial y_L}{\partial b_L} = \sigma' \Rightarrow \frac{\partial y_L}{\partial b_L} = \text{diag}(\sigma') \cdot \mathbf{1}_{n_L}, \quad \frac{\partial y_L}{\partial b_L} = \frac{\partial y_L}{\partial y_L} \frac{\partial y_L}{\partial b_L} = v_L \mathbf{1}_{n_L}$$

$$\textcircled{4} \frac{\partial y_L}{\partial w_\ell} = \frac{\partial y_L}{\partial y_\ell} \frac{\partial y_\ell}{\partial w_\ell} = \frac{\partial y_L}{\partial y_\ell} \frac{\partial y_\ell}{\partial y_{\ell-1}} \frac{\partial y_{\ell-1}}{\partial w_\ell} = \frac{\partial y_L}{\partial y_\ell} \text{diag}(\sigma') \frac{\partial y_{\ell-1}}{\partial w_\ell} = v_\ell \frac{\partial y_{\ell-1}}{\partial w_\ell} \quad \text{이전}$$

$$\left(\frac{\partial y_{\ell-1}}{\partial w_\ell} \right)_k = \frac{\partial (A_{w_\ell} y_{\ell-1} + b_\ell \mathbf{1}_{n_\ell})_k}{\partial (w_\ell)_j} = \frac{\partial (\sum_{m=1}^{f_\ell} (w_\ell)_{m,j} (y_{\ell-1})_{k+m-1} + b_\ell)}{\partial (w_\ell)_j} = (y_{\ell-1})_{k+j-1} \quad (1 \leq j \leq f_\ell) \text{ 이고}$$

$$\left(\frac{\partial y_L}{\partial w_\ell} \right)_j = \sum_{k=1}^{n_\ell} (v_L^\top)_k (y_{\ell-1})_{k+j-1} \quad (1 \leq j \leq f_\ell) \text{ 이고, } (C_{v_L^\top} y_{\ell-1})_j = \sum_{k=1}^{n_\ell} (y_{\ell-1})_{j+k-1} (v_L^\top)_k \quad (1 \leq j \leq f_\ell) \text{ 이고}$$

$$\frac{\partial y_L}{\partial w_\ell} = (C_{v_L^\top} y_{\ell-1})^\top$$

MATHDNN_HW7

November 21, 2021

Problem 7

```
[1]: import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets
import torch.optim as optim
from torchvision.transforms import transforms
from torchvision.utils import save_image

import numpy as np
import matplotlib.pyplot as plt
```

```
[2]: lr = 0.001
batch_size = 100
epochs = 10
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Step I

```
[3]: # MNIST dataset
dataset = datasets.MNIST(root='./mnist_data/',
                          train=True,
                          transform=transforms.ToTensor(),
                          download=True)

train_dataset, validation_dataset = torch.utils.data.random_split(dataset, [
    ↪ 50000, 10000])

test_dataset = datasets.MNIST(root='./mnist_data/',
                              train=False,
                              transform=transforms.ToTensor())

# KMNIST dataset, only need test dataset
anomaly_dataset = datasets.KMNIST(root='./kmnist_data/',
                                  train=False,
                                  transform=transforms.ToTensor(),
```

```
download=True)
```

```
# print(len(train_dataset)) # 50000
# print(len(validation_dataset)) # 10000
# print(len(test_dataset)) # 10000
# print(len(anomaly_dataset)) # 10000
```

```
Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz
Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz to
./mnist_data/MNIST/raw/train-images-idx3-ubyte.gz
```

```
0%|          | 0/9912422 [00:00<?, ?it/s]
```

```
Extracting ./mnist_data/MNIST/raw/train-images-idx3-ubyte.gz to
./mnist_data/MNIST/raw
```

```
Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz
Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz to
./mnist_data/MNIST/raw/train-labels-idx1-ubyte.gz
```

```
0%|          | 0/28881 [00:00<?, ?it/s]
```

```
Extracting ./mnist_data/MNIST/raw/train-labels-idx1-ubyte.gz to
./mnist_data/MNIST/raw
```

```
Downloading http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz
Downloading http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz to
./mnist_data/MNIST/raw/t10k-images-idx3-ubyte.gz
```

```
0%|          | 0/1648877 [00:00<?, ?it/s]
```

```
Extracting ./mnist_data/MNIST/raw/t10k-images-idx3-ubyte.gz to
./mnist_data/MNIST/raw
```

```
Downloading http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz
Downloading http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz to
./mnist_data/MNIST/raw/t10k-labels-idx1-ubyte.gz
```

```
0%|          | 0/4542 [00:00<?, ?it/s]
```

```
Extracting ./mnist_data/MNIST/raw/t10k-labels-idx1-ubyte.gz to
./mnist_data/MNIST/raw
```

```
Downloading http://codh.rois.ac.jp/kmnist/dataset/kmnist/train-images-
idx3-ubyte.gz
Downloading http://codh.rois.ac.jp/kmnist/dataset/kmnist/train-images-
idx3-ubyte.gz to ./kmnist_data/KMNIST/raw/train-images-idx3-ubyte.gz
```

```
0%|          | 0/18165135 [00:00<?, ?it/s]
```

```
Extracting ./kmnist_data/KMNIST/raw/train-images-idx3-ubyte.gz to
./kmnist_data/KMNIST/raw
```

Downloading http://codh.rois.ac.jp/kmnist/dataset/kmnist/train-labels-idx1-ubyte.gz

Downloading http://codh.rois.ac.jp/kmnist/dataset/kmnist/train-labels-idx1-ubyte.gz to ./kmnist_data/KMNIST/raw/train-labels-idx1-ubyte.gz

0%| | 0/29497 [00:00<?, ?it/s]

Extracting ./kmnist_data/KMNIST/raw/train-labels-idx1-ubyte.gz to ./kmnist_data/KMNIST/raw

Downloading http://codh.rois.ac.jp/kmnist/dataset/kmnist/t10k-images-idx3-ubyte.gz

Downloading http://codh.rois.ac.jp/kmnist/dataset/kmnist/t10k-images-idx3-ubyte.gz to ./kmnist_data/KMNIST/raw/t10k-images-idx3-ubyte.gz

0%| | 0/3041136 [00:00<?, ?it/s]

Extracting ./kmnist_data/KMNIST/raw/t10k-images-idx3-ubyte.gz to ./kmnist_data/KMNIST/raw

Downloading http://codh.rois.ac.jp/kmnist/dataset/kmnist/t10k-labels-idx1-ubyte.gz

Downloading http://codh.rois.ac.jp/kmnist/dataset/kmnist/t10k-labels-idx1-ubyte.gz to ./kmnist_data/KMNIST/raw/t10k-labels-idx1-ubyte.gz

0%| | 0/5120 [00:00<?, ?it/s]

Extracting ./kmnist_data/KMNIST/raw/t10k-labels-idx1-ubyte.gz to ./kmnist_data/KMNIST/raw

Step II : AutoEncoder

```
[4]: # Define Encoder
class Encoder(nn.Module):
    def __init__(self):
        super(Encoder, self).__init__()
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 32)
    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        z = F.relu(self.fc3(x))
        return z

# Define Decoder
class Decoder(nn.Module):
    def __init__(self):
        super(Decoder, self).__init__()
```

```

self.fc1 = nn.Linear(32, 128)
self.fc2 = nn.Linear(128, 256)
self.fc3 = nn.Linear(256, 784)
def forward(self, z):
    z = F.relu(self.fc1(z))
    z = F.relu(self.fc2(z))
    x = F.sigmoid(self.fc3(z)) # to make output's pixels are 0~1
    x = x.view(x.size(0), 1, 28, 28)
    return x

```

Step III : Instantiate model & define loss and optimizer

```

[5]: enc = Encoder().to(device)
    dec = Decoder().to(device)
    loss_function = nn.MSELoss()
    optimizer = optim.Adam(list(enc.parameters()) + list(dec.parameters()), lr=lr)

```

Step IV : Training

```

[6]: train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
    ↪batch_size=batch_size, shuffle=True)

train_loss_list = []

import time
start = time.time()
for epoch in range(epochs) :
    print("{}th epoch starting.".format(epoch))
    enc.train()
    dec.train()
    for batch, (images, _) in enumerate(train_loader) :
        images = images.to(device)
        z = enc(images)
        reconstructed_images = dec(z)

        optimizer.zero_grad()
        train_loss = loss_function(images, reconstructed_images)
        train_loss.backward()
        train_loss_list.append(train_loss.item())

    optimizer.step()

    print(f"[Epoch {epoch:3d}] Processing batch #{batch:3d} reconstruction_
    ↪loss: {train_loss.item():.6f}", end='\r')
end = time.time()
print("Time ellapsed in training is: {}".format(end - start))

```

```

# plotting train loss
plt.plot(range(1,len(train_loss_list)+1), train_loss_list, 'r', label='Training_
↳loss')
plt.title('Training loss')
plt.xlabel('Iterations')
plt.ylabel('Loss')
plt.legend()
plt.savefig('loss.png')

enc.eval()
dec.eval()

```

0th epoch starting.

/usr/local/lib/python3.7/dist-packages/torch/nn/functional.py:1806: UserWarning:
nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.

warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid
instead.")

1th epoch starting.

2th epoch starting.

3th epoch starting.

4th epoch starting.

5th epoch starting.

6th epoch starting.

7th epoch starting.

8th epoch starting.

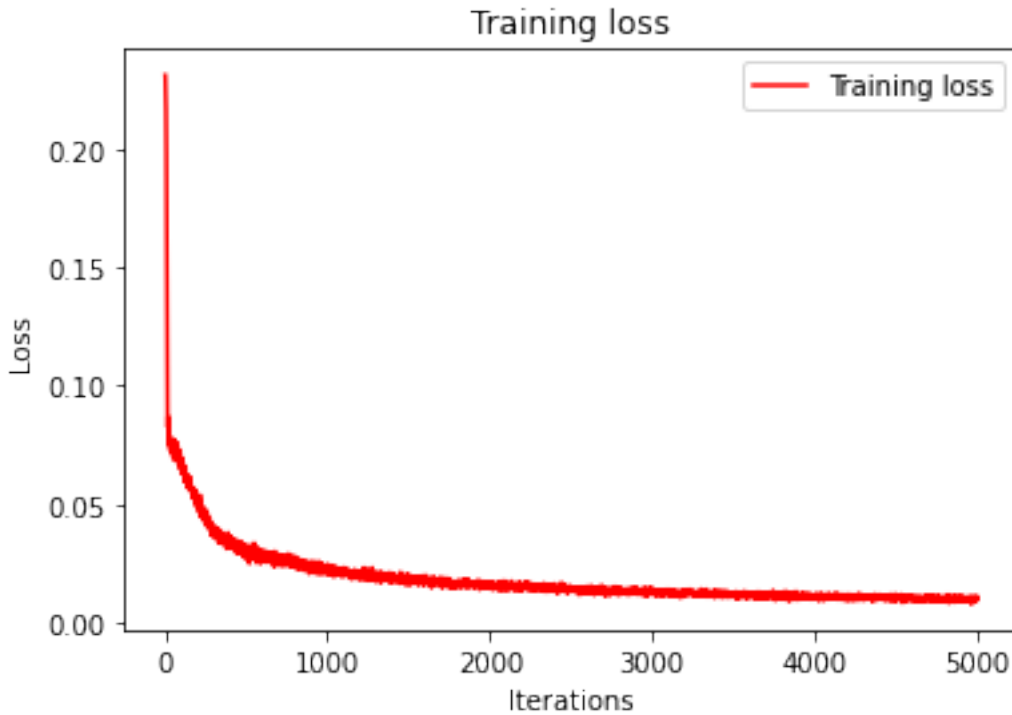
9th epoch starting.

Time ellapsed in training is: 99.13060545921326

```

[6]: Decoder(
  (fc1): Linear(in_features=32, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=256, bias=True)
  (fc3): Linear(in_features=256, out_features=784, bias=True)
)

```



Step V : Calculate standard deviation by using validation set

```
[7]: validation_loader = torch.utils.data.DataLoader(dataset=validation_dataset,
    ↪ batch_size=batch_size)

scores = []

for images, _ in validation_loader:
    images = images.to(device)
    z = enc(images)
    reconstructed_images = dec(z)
    score = loss_function(images, reconstructed_images)
    scores.append(score.item())

scores = torch.Tensor(scores)
mean = torch.mean(scores)
std = torch.std(scores)

threshold = mean + 3 * std
print("threshold: ", threshold.tolist())
```

```
/usr/local/lib/python3.7/dist-packages/torch/nn/functional.py:1806: UserWarning:
nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.
  warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid
```

instead.")

threshold: 0.012076077982783318

Step VI : Anomaly detection (MNIST)

```
[8]: exact = 0
dismatch = 0

test_loader = torch.utils.data.DataLoader(dataset=test_dataset,
    ↪batch_size=batch_size)

for images, _ in test_loader:
    images = images.to(device)
    z = enc(images)
    reconstructed_images = dec(z)
    score = loss_function(images, reconstructed_images)

    if score > threshold:
        dismatch += 1
    else:
        exact += 1

print(f"Type I error rate(MNIST Dataset) is {dismatch/(dismatch+exact)}")
```

/usr/local/lib/python3.7/dist-packages/torch/nn/functional.py:1806: UserWarning:
nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.

warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid
instead.")

Type I error rate(MNIST Dataset) is 0.03

Step VII : Anomaly detection (KMNIST)

```
[9]: exact = 0
dismatch = 0

anomaly_loader = torch.utils.data.DataLoader(dataset=anomaly_dataset,
    ↪batch_size=batch_size)

for images, _ in anomaly_loader:
    images = images.to(device)
    z = enc(images)
    reconstructed_images = dec(z)
    score = loss_function(images, reconstructed_images)

    if score > threshold:
        dismatch += 1
    else:
        exact += 1
```

```
print(f"Type II error rate(KMNIST Dataset) is {exact/(dismatch+exact)}")
```

```
/usr/local/lib/python3.7/dist-packages/torch/nn/functional.py:1806: UserWarning:  
nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.
```

```
    warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid  
instead.")
```

```
Type II error rate(KMNIST Dataset) is 0.0
```